

PixHomology Benchmark: User and Reproducibility Guide

PixHomology Development Team

November 6, 2025

Contents

1	Introduction	2
2	PixHomology: Core Functionality	2
2.1	Basic Workflow	2
2.2	Example Usage	2
2.3	(.	2
3	Repository Overview	3
3.1	Main Structure	3
4	Installation	3
4.1	Creating the Environment	3
4.2	Verification	3
5	Using PHBenchmark	3
5.1	Dataset Download	3
5.2	Running Benchmarks	4
5.3	Collecting and Reporting	4
6	End-to-End MNIST Example	4
7	Running on SLURM	4
8	Software Installed	5
9	Reproducibility Notes	5

1 Introduction

PixHomology is a C++/pybind11 library for computing persistent homology on 2D images. It provides an efficient and memory-aware implementation for computing 0-dimensional and 1-dimensional persistence diagrams from grayscale or binary images represented as NumPy 2D arrays. The accompanying **PHBenchmark** package automates the benchmarking process across multiple persistent homology backends (PixHomology, Cripser, GUDHI, Ripser, TTK) and datasets.

This document explains:

- The internal logic and basic usage of PixHomology.
- How to compute persistent homology from NumPy arrays.
- How to install and use the PHBenchmark repository for reproducing experiments and reports.

2 PixHomology: Core Functionality

2.1 Basic Workflow

PixHomology converts a 2D NumPy array `img` into a complex and computes persistence diagrams through lower-star filtrations.

The typical steps are:

1. Import the library and provide a 2D NumPy array.
2. Choose the homology dimension (0 or 1).
3. Compute the persistence diagram.

2.2 Example Usage

Listing 1: Basic persistent homology computation using PixHomology

```
import numpy as np
import pixhomology as px

# Example: create a synthetic 2D image
img = np.random.rand(28, 28)

# Compute 0- and 1-dimensional persistence diagrams
ph0 = px.computePH(img, maxdim=0)
ph0, ph1 = px.computePH(img, maxdim=1)

print("H0-diagram:", ph0)
print("H1-diagram:", ph1)
```

The output is a list of (birth, death) pairs for each topological feature. The function automatically handles grayscale-to-filtration conversion using lower-star filtration rules.

2.3 Documentation and API

For a complete description of all functionalities and the full API of PixHomology, please refer to the official library documentation available at:

<https://riccardoc95.github.io/PixHomology/>

3 Repository Overview

The PHBenchmark project wraps PixHomology and other libraries into a unified benchmarking toolchain.

3.1 Main Structure

```
PHBenchmark/
  environment.yml          # Conda environment for full reproducibility
  run_all.sbatch           # SLURM array job for distributed benchmarking
  phbenchmark/
    app.py                 # CLI entry point (Typer)
    download.py            # dataset fetching/extraction
    run.py                 # executes benchmarks via /usr/bin/time
    collect.py             # aggregates CSV -> JSON summaries
    report.py              # generates plots and LaTeX tables
    tests/                 # wrappers for each method (pixh, gudhi, etc.)
  PixHomology/             # local source for PixHomology library
```

4 Installation

The repository includes an `environment.yml` file that builds a complete reproducible environment.

4.1 Creating the Environment

```
conda env create -f environment.yml
conda activate phbenchmark
```

This installs:

- Python 3.11 and Topology ToolKit (TTK)
- Persistent homology libraries: Ripser, Cripser, GUDHI
- PixHomology (editable install from local source)
- PHBenchmark CLI

4.2 Verification

```
phbenchmark --help
python -c "import pixhomology as px; print('PixHomology OK', px.__version__)"
```

5 Using PHBenchmark

5.1 Dataset Download

```
phbenchmark download --datasets_dir datasets --dataset mnist
```

Available datasets: MNIST, CIFAR10, ImageNet, DIV2K, Kather. Images are expanded to `datasets/<name>_npz/`.

5.2 Running Benchmarks

Each run measures runtime and peak memory usage.

```
phbenchmark run \
  --method pixh \
  --dataset datasets/mnist.npy \
  --maxdim 1 \
  --output results/pixh/mnist/dim1.csv
```

5.3 Collecting and Reporting

```
phbenchmark collect \
  --results results \
  --output results/benchmarks.json \
  --summary results/benchmarks_summary.json
```

```
phbenchmark report \
  --summary results/benchmarks_summary.json \
  --plots-dir results/plots \
  --latex-output results/benchmark_table.tex
```

Outputs include detailed plots and a LaTeX table summarizing average runtime and memory per method and dataset.

6 End-to-End MNIST Example

```
conda activate phbenchmark
phbenchmark download --datasets-dir datasets --dataset mnist
```

```
for m in pixh cripsier gudhi ripser ttk; do
  for k in 0 1; do
    out="results/${m}/mnist/dim${k}.csv"
    mkdir -p $(dirname $out)
    phbenchmark run --method $m --dataset datasets/mnist.npy \
      --maxdim $k --output $out
  done
done
```

```
phbenchmark collect --results results --output results/benchmarks.json \
  --summary results/benchmarks_summary.json
phbenchmark report --summary results/benchmarks_summary.json \
  --plots-dir results/plots --latex-output results/benchmark_table.tex
```

7 Running on SLURM

The provided `run_all.sbatch` automates all runs as a SLURM job array.

```
sbatch --array=0-<N-1> run_all.sbatch
```

Edit the file to configure datasets, methods, and dimensions. Logs are stored under `logs/`.

8 Software Installed

- **PixHomology** - This repository (C++/pybind11)
- **Cripser** - https://github.com/shizuo-kaji/CubicalRipser_3dim
- **GUDHI** - <https://gudhi.inria.fr/>
- **Ripser.py** - <https://ripser.scikit-tda.org/en/latest/>
- **Topology ToolKit (TTK)** - <https://topology-tool-kit.github.io/>

General Python utilities: `typer`, `tqdm`, `pandas`, `matplotlib`, `seaborn`, `jinja2`.

9 Reproducibility Notes

- Versions pinned in `environment.yml`.
- Runtime and memory measured using `/usr/bin/time`.
- CPU threads controlled via `OMP_NUM_THREADS`.